



WESTCLIFF
UNIVERSITY
Educate. Inspire. Empower.

REACT Review

Module 5 - Week 2 Day 3



Agenda

1. What are controlled forms, when to use and examples
2. What are uncontrolled forms, when to use and examples Setting up React
3. What is React-Router, purpose and how to use
4. What is SPA, purpose
5. What is Code Splitting, why and how
6. Typescript with React
7. In-Class Demo Exercise
8. Homework



Controlled vs Uncontrolled Components

Controlled component, the form data is handled by the state within the component. The state within the component serves as “the single source of truth” for the input elements that are rendered by the component.

Uncontrolled components act more like traditional HTML form elements. The data for each input element is stored in the DOM, not in the component. Instead of writing an event handler for all of your state updates, you use a **ref** to retrieve values from the DOM.

Refs provide a way to access DOM nodes or React elements created in the render method.

What are controlled forms, when to use and examples



What are controlled forms?

HTML **Form** elements work a little bit differently from other DOM elements in React, because form elements naturally keep some internal state.

This form in plain HTML accepts a single “name”

```
<form>  
  <label>  
    Name:  
    <input type="text" name="name" />  
  </label>  
  <input type="submit" value="Submit" />  
</form>
```



What are controlled forms?

This form in previous slide has the default HTML form behavior of browsing to a new page when the user submits the form.

If you want this behavior in React, it just works. But in most cases, it's convenient to have a JavaScript function that handles the submission of the form and has access to the data that the user entered into the form. The standard way to achieve this is with a technique called “**controlled components**”.

```
<form>
  <label>
    Name:
    <input type="text" name="name" />
  </label>
  <input type="submit" value="Submit" />
</form>
```



What are controlled forms?

In HTML, form elements such as `<input>`, `<textarea>`, and `<select>` typically maintain their own state and update it based on user input.

In React, mutable state is typically kept in the **state** property of components, and only updated with **setState()**.

The React component that renders a form also controls what happens in that form on subsequent user input. An input form element whose value is controlled by React in this way is called a “**controlled component**”.



What are controlled forms?

A React form from previous example would look like this:

On submit, run
handleSubmit
method

```
class NameForm extends React.Component {
  constructor(props) {
    super(props);
    this.state = {value: ''};

    this.handleChange = this.handleChange.bind(this);
    this.handleSubmit = this.handleSubmit.bind(this);
  }

  handleChange(event) {
    this.setState({value: event.target.value});
  }

  handleSubmit(event) {
    alert('A name was submitted: ' + this.state.value);
    event.preventDefault();
  }

  render() {
    return (
      <form onSubmit={this.handleSubmit}>
        <label>
          Name:
          <input type="text" value={this.state.value} onChange={this.handleChange} />
        </label>
        <input type="submit" value="Submit" />
      </form>
    );
  }
}

ReactDOM.render(
  <NameForm />,
  document.getElementById('root')
);
```




What are controlled forms?

The value attribute is set on our form element, the displayed value will always be **this.state.value**, making the React state the source of truth.

handleChange runs on every keystroke to update the React state, the displayed value will update as the user types.

With a **controlled component**, the input's value is always driven by the **React state**.

```
class NameForm extends React.Component {
  constructor(props) {
    super(props);
    this.state = {value: ''};

    this.handleChange = this.handleChange.bind(this);
    this.handleSubmit = this.handleSubmit.bind(this);
  }

  handleChange(event) {
    this.setState({value: event.target.value});
  }

  handleSubmit(event) {
    alert('A name was submitted: ' + this.state.value);
    event.preventDefault();
  }

  render() {
    return (
      <form onSubmit={this.handleSubmit}>
        <label>
          Name:
          <input type="text" value={this.state.value} onChange={this.handleChange} />
        </label>
        <input type="submit" value="Submit" />
      </form>
    );
  }
}

ReactDOM.render(
  <NameForm />,
  document.getElementById('root')
);
```



What are controlled forms?

a `<textarea>` element defines its text by its children:

In React, a `<textarea>` uses a `value` attribute instead. This way, a form using a `<textarea>` can be written very similarly to a form that uses a single-line input:

```
<textarea>
  Hello, this is text area
</textarea>

render() {
  return (
    <form onSubmit={this.handleSubmit}>
      <label>
        Essay:
        <textarea value={this.state.value}
onChange={this.handleChange} />      </label>
      <input type="submit" value="Submit" />
    </form>
  );
}
```



What are controlled forms?

HTML, `<select>` creates a drop-down list.

The **ReactJS** option is initially selected, because of the `selected` attribute. React, instead of using this `selected` attribute, uses a `value` attribute on the root `select` tag.

This is more convenient in a controlled component because you only need to update it in one place.

```
<select>
  <option value="expressjs">ExpressJS</option>
  <option value="vuejs">VueJS</option>
  <option selected value="reactjs">ReactJS</option>
  <option value="mongo">Mongo</option>
</select>
```



What are controlled forms?

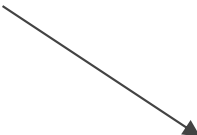
```
constructor(props) {  
  super(props);  
  this.state = {value: 'reactjs'};  
  this.handleChange = this.handleChange.bind(this);  
  this.handleSubmit = this.handleSubmit.bind(this);  
}  
  
handleChange(event) {  
  this.setState({value: event.target.value});  
}
```

```
render() {  
  return (  
    <form onSubmit={this.handleSubmit}>  
      <label>  
        Select your Bootcamp:  
        <select value={this.state.value} onChange={this.handleChange}>  
          <option value="expressjs">ExpressJS</option>  
          <option value="vuejs">VueJS</option>  
          <option value="reactjs">ReactJS</option>  
          <option value="mongo">Mongo</option>  
        </select>  
      </label>  
      <input type="submit" value="Submit" />  
    </form>  
  );  
}
```



What are controlled forms?

an `<input type="file">` lets the user choose one or more files from their device storage to be uploaded to a server or manipulated by JavaScript via the **File API**.



```
<input type="file" />
```



What are controlled forms?

Handling Multiple Inputs

To handle multiple controlled input elements, you can add a name attribute to each element and let the handler function choose what to do based on the value of **event.target.name**.

setState() automatically merges a partial state into the current state, we only needed to call it with the changed parts

we used the ES6 computed property name syntax to update the state key corresponding to the given input name

```
handleInputChange(event) {  
  const target = event.target;  
  const value = target.type === 'checkbox' ? target.checked : target.value;  
  const name = target.name;  
  this.setState({  
    [name]: value  });  
}
```



What are controlled forms?

Input Null Value

the value prop on a controlled component prevents the user from changing the input unless you desire so. If you've specified a value but the input is still editable, you may have accidentally set value to undefined or null.

```
ReactDOM.render(<input value="hi" />, mountNode);  
  
setTimeout(function() {  
  ReactDOM.render(<input value={null} />, mountNode);  
}, 1000);
```

The input is locked at first but becomes editable after a short delay



What are controlled forms?

It can sometimes be tedious to use controlled components, because you need to write an event handler for every way your data can change and pipe all of the input state through a React component.

This can become particularly annoying when you are converting a preexisting codebase to React, or integrating a React application with a non-React library. In these scenarios we use uncontrolled components.

What are uncontrolled forms, when use and examples

Setting up React



What are uncontrolled forms?

It is recommend to use controlled components to implement forms. In a controlled component, form data is handled by a React component. The alternative is uncontrolled components, where form data is handled by the DOM itself.

To write an uncontrolled component, instead of writing an event handler for every state update, you can use a ref to get form values from the DOM.



What are uncontrolled forms?

this code accepts a single name in an uncontrolled component

An uncontrolled component keeps the source of truth in the DOM, it is sometimes easier to integrate React and non-React code when using uncontrolled components.

```
class NameForm extends React.Component {
  constructor(props) {
    super(props);
    this.handleSubmit = this.handleSubmit.bind(this);
    this.input = React.createRef();
  }

  handleSubmit(event) {
    alert('A name was submitted: ' + this.input.current.value);
    event.preventDefault();
  }

  render() {
    return (
      <form onSubmit={this.handleSubmit}>
        <label>
          Name:
          <input type="text" ref={this.input} />
        </label>
        <input type="submit" value="Submit" />
      </form>
    );
  }
}
```



What are uncontrolled forms?

Default Values

React rendering lifecycle, the value attribute on form elements will override the value in the DOM. With an uncontrolled component, you often want React to specify the initial value, but leave subsequent updates uncontrolled. To handle this case, you can specify a `defaultValue` attribute instead of `value`. Changing the value of `defaultValue` attribute after a component has mounted will not cause any update of the value in the DOM

```
render() {  
  return (  
    <form onSubmit={this.handleSubmit}>  
      <label>  
        Name:  
        <input  
          defaultValue="Bob"  
          type="text"  
          ref={this.input} />  
      </label>  
      <input type="submit" value="Submit" />  
    </form>  
  );  
}
```

`<input type="checkbox">` and `<input type="radio">` support **defaultChecked**, and `<select>` and `<textarea>` supports **defaultValue**.



What are uncontrolled forms?

Input File

In React, an `<input type="file" />` is always an uncontrolled component because its value can only be set by a user, and not programmatically.

You should use the File API to interact with the files

```
constructor(props) {
  super(props);
  this.handleSubmit = this.handleSubmit.bind(this);
  this.fileInput = React.createRef();
}

handleSubmit(event) {
  event.preventDefault();
  alert(
    `Selected file - ${this.fileInput.current.files[0].name}`
  );
}

render() {
  return (
    <form onSubmit={this.handleSubmit}>
      <label>
        Upload file:
        <input type="file" ref={this.fileInput} />
      </label>
      <br />
      <button type="submit">Submit</button>
    </form>
  );
}
```



REF and DOM

Ref and DOM

Refs provide a way to access DOM nodes or React elements created in the render method.

React dataflow, props are the only way that parent components interact with their children. To modify a child, you re-render it with new props.

There are a few cases where you need to imperatively modify a child outside of the typical dataflow. The child to be modified could be an instance of a React component, or it could be a DOM element



REF and DOM

When to Use Refs

Some use cases for refs:

- Managing focus, text selection, or media playback.
- Triggering imperative animations.
- Integrating with third-party DOM libraries.

Avoid using refs for anything that can be done declaratively.

Instead of exposing **open()** and **close()** methods on a Dialog component, pass an **isOpen** prop to it.



REF and DOM

Creating Refs

Refs are created using `React.createRef()` and attached to React elements via the `ref` attribute. Refs are commonly assigned to an instance property when a component is constructed so they can be referenced throughout the component.

```
class MyComponent extends React.Component {  
  constructor(props) {  
    super(props);  
    this.myRef = React.createRef();  
  }  
  render() {  
    return <div ref={this.myRef} />;  
  }  
}
```




REF and DOM

Accessing Refs

When a ref is passed to an element in render, a reference to the node becomes accessible at the current attribute of the ref.

```
const node = this.myRef.current;
```

The value of the ref differs depending on the type of the node:

- When the ref attribute is used on an HTML element, the ref created in the constructor with `React.createRef()` receives the underlying DOM element
- When the ref attribute is used on a custom class component, the ref object receives the mounted instance of the component as its current.
- You may not use the ref attribute on function components because they don't have instances.

What is React-Router, purpose and how to use



What are React-Router?

React Router is the routing library for React. It is not developed by facebook, it is a third-party solution. When you need to navigate through a React application with multiple views, you'll need a router to manage the URLs.

React Router takes care of that, keeping your application UI and the URL in sync.

Routing is the process of keeping the browser URL in sync with what's being rendered on the page. React Router lets you handle routing declaratively. The declarative routing approach allows you to control the data flow in your application.

You can place your `<Route>` component anywhere you want your route to be rendered. Since `<Route>`, `<Link>` and all the other React Router APIs that we'll be dealing with are just components, you can easily get up and running with routing in React.

```
<Route path="/about">  
  <About />  
</Route>
```



What are React-Router?

React Router offers a lot of features such as:

- Basic navigational routing
- Nested routing
- Nested routing with path parameters
- Protected routing

We will take a look at some of these features today.



What are React-Router?

To download React-Router we use NPX and react app.

```
npx create-react-app react-router-dom
```

Download and
create React app

Download react-router DOM
package

The React Router library comprises three packages: [react-router](#), [react-router-dom](#), and [react-router-native](#). The core package for the router is [react-router](#), the other two are environment specific. You should use [react-router-dom](#) if you're building a website, and [react-router-native](#) if you're in a mobile app development environment using React Native.



React-Router - BrowserRouter

<BrowserRouter>

A **<Router>** that uses the HTML5 history API (pushState, replaceState and the popstate event) to keep your UI in sync with the URL.

1. `basename: string`

The base URL for all locations. If your app is served from a sub-directory on your server, you'll want to set this to the sub-directory. A properly formatted `basename` should have a leading slash, but no trailing slash.

```
<BrowserRouter
  basename={optionalString}
  forceRefresh={optionalBool}
  getUserConfirmation={optionalFunc}
  keyLength={optionalNumber}
>
  <App />
</BrowserRouter>
```



React-Router - BrowserRouter

2. forceRefresh: bool

If true the router will use full page refreshes on page navigation. You may want to use this to imitate the way a traditional server-rendered app would work with full page refreshes between page navigation.

```
<BrowserRouter forceRefresh={true} />
```

3. getUserConfirmation: func

A function to use to confirm navigation. Defaults to using window.confirm.

4. keyLength: number

```
<BrowserRouter keyLength={12} />
```

The length of location.key. Defaults to 6.

children: node

The child elements to render.



React-Router - Link

`<Link>`

Provides declarative, accessible navigation around your application.

1. `to`: string

```
<Link to="/about">About</Link>
```

A string representation of the Link location, created by concatenating the location pathname, search, and hash properties.

2. `to`: object

```
<Link to="/courses?sort=name" />
```

An object that can have any of the following properties:

- `pathname`: A string representing the path to link to.
- `search`: A string representation of query parameters.
- `hash`: A hash to put in the URL, e.g. `#a-hash`.
- `state`: State to persist to the location.



React-Router - Link

3. to: function

```
<Link to={location => ({ ...location, pathname: "/courses" })} />
```

A function to which current location is passed as an argument and which should return location representation as a string or as an object

4. replace: bool

```
<Link to="/courses" replace />
```

When true, clicking the link will replace the current entry in the history stack instead of adding a new one.

5. innerRef: function / RefObject

forward the ref to the underlying `<a>`.
Use a normal ref instead.

```
<Link  
  to="/"  
  innerRef={node => {  
    // `node` refers to the mounted DOM element  
    // or null when unmounted  
  }}  
/>
```



React-Router - Switch

<Switch>

Renders the first child <Route> or <Redirect> that matches the location.

How is this different than just using a bunch of <Route>s?

<Switch> is unique in that it renders a route exclusively. In contrast, every <Route> that matches the location renders inclusively.

If the URL is /about, then <About>, <User>, and <NoMatch> will all render because they all match the path. This is by design, allowing us to compose <Route>s into our apps in many ways

```
import { Route } from "react-router";

let routes = (
  <div>
    <Route path="/about">
      <About />
    </Route>
    <Route path="/:user">
      <User />
    </Route>
    <Route>
      <NoMatch />
    </Route>
  </div>
);
```



React-Router - Switch

If we're at /about, `<Switch>` will start looking for a matching `<Route>`.
`<Route path="/about"/>` will match and `<Switch>` will stop looking for matches and render `<About>`

```
import { Route, Switch } from "react-router";

let routes = (
  <Switch>
    <Route exact path="/">
      <Home />
    </Route>
    <Route path="/about">
      <About />
    </Route>
    <Route path="/:user">
      <User />
    </Route>
    <Route>
      <NoMatch />
    </Route>
  </Switch>
);
```



React-Router - Switch

1. location: object

A location object to be used for matching children elements instead of the current history location (usually the current browser URL).

2. children: node

All children of a `<Switch>` should be `<Route>` or `<Redirect>` elements. Only the first child to match the current location will be rendered.

`<Route>` elements are matched using their `path` prop and `<Redirect>` elements are matched using their `from` prop. A `<Route>` with no `path` prop or a `<Redirect>` with no `from` prop will always match the current location.



React-Router - Route

`<Route>`

The Route component is perhaps the most important component in React Router to understand and learn to use well. Its most basic responsibility is to render some UI when its path matches the current URL.

Route render methods

The method of rendering something with a `<Route>` is to use children elements. There are, however, a few other methods you can use to render something with a `<Route>`.

- `<Route component>`
- `<Route render>`
- `<Route children>` function

You should use only one of these props on a given `<Route>`.



React-Router - Route

Route props

All three render methods will be passed the same three route props

- match
- location
- history

component

A React component to render only when the location matches. It will be rendered with route props.

```
import { BrowserRouter as Router, Route } from "react-router-dom";

ReactDOM.render(
  <Router>
    <div>
      <Route exact path="/">
        <Home />
      </Route>
      <Route path="/news">
        <NewsFeed />
      </Route>
    </div>
  </Router>,
  node
);

<div>
  <Home />
</div>

<div>
  <NewsFeed />
</div>
```

What is SPA, purpose



What are SPA?

React is a popular library for creating **single-page applications (SPAs)** that are rendered on the client side.

An SPA has multiple views (pages), and unlike conventional multi-page apps, navigating through these views shouldn't result in the entire page being reloaded. Instead, we want the views to be rendered inline within the current page. The end user, who's accustomed to multi-page apps can expects SPA features such as:

- A View have a URL that is specifies that view. This allows user to bookmark the URL for reference if needed. For example: www.domain.com/cats
- The browser's back and forward button should work as expected or mouse shortcuts.
- Dynamically generated nested views should have a URL of their own, such as domain.com/cats/abyssinian/12, where 12 is the cat type ID.



What are SPA?

Examples of Single Page Applications

Some notable examples:

- Gmail
- Google Maps
- Facebook
- Twitter
- Google Drive



What are SPA?

Pros of SPAs

- An SPA is faster, instead of waiting for the client-server communication, and wait for the browser to re-render the page, you can now have instant feedback. This is the responsibility of the application maker, but you can have transitions and spinners and any kind of UX improvement that is certainly better than the traditional workflow.
- The server will consume less resources because you can focus on providing an efficient API instead of building the layouts server-side.
- This makes it ideal if you also build a mobile app on top of the API, as you can completely reuse your existing server-side code.
- Single Page Applications are easy to transform into Progressive Web Apps, which in turn enables you to provide local caching and to support offline experiences for your services.



What are SPA?

Pros of SPAs (continued)

- SPAs are best used when there is no need for SEO.
- Search engines, still have trouble indexing sites built with an SPA approach rather than the traditional server-rendered pages. This is the case for blogs.
- You need to pay a lot more attention to possible memory leaks. if in the past your page had a lifespan that was counted in minutes, now an SPA might stay open for hours at a time and if there is any memory issue that's going to increase the browser memory usage by a lot. This can cause an issue with UI/UX.
- SPAs are great when working in teams. Backend developers can just focus on the API, and frontend developers can focus on creating the best user experience.



What are SPA?

Cons of SPAs

- Single Page Apps rely heavily on JavaScript.
- SPAs can make using an application running on low power devices a poor experience in terms of speed.
- some of your visitors might just have JavaScript disabled
- You need to consider accessibility for anything you build.

What is Code Splitting, why and how



What is Code Splitting?

Code Splitting also known as **bundling** the code. **Bundling** is great, but as your app grows, your bundle will grow too. Especially if you are including large third-party libraries. You need to keep an eye on the code you are including in your bundle so that you don't accidentally make it so large that your app takes a long time to load.

To avoid winding up with a large bundle, it's good to get ahead of the problem and start "splitting" your bundle. Code-Splitting is a feature supported by bundlers like Webpack, Rollup and Browserify (via factor-bundle) which can create multiple bundles that can be dynamically loaded at runtime.

Code-splitting your app can help you "lazy-load" just the things that are currently needed by the user, which can dramatically improve the performance of your app. While you haven't reduced the overall amount of code in your app, you've avoided loading code that the user may never need, and reduced the amount of code needed during the initial load.



What is Code Splitting?

Bundling is the process of following imported files and merging them into a single file: a “bundle”. This bundle can then be included on a webpage to load an entire app at once.

App:

```
// app.js
import { add } from './math.js';

console.log(add(16, 26)); // 42
```

```
// math.js
export function add(a, b) {
  return a + b;
}
```

Bundle:

```
function add(a, b) {
  return a + b;
}

console.log(add(16, 26)); // 42
```



What is Code Splitting?

`import()`

The best way to introduce code-splitting into your app is through the dynamic `import()` syntax.

Before:

```
import { add } from './math';  
console.log(add(16, 26));
```

After:

```
import("./math").then(math => {  
  console.log(math.add(16, 26));  
});
```

When Webpack comes across this syntax, it automatically starts code-splitting your app. If you're using Create React App, this is already configured for you and you can start using it



What is Code Splitting?

React.Lazy

The `React.lazy` function lets you render a dynamic import as a regular component.

Before:

```
import OtherComponent from './OtherComponent';
```

After:

```
const OtherComponent = React.lazy(() =>  
  import('./OtherComponent'));
```

This will automatically load the bundle containing the `OtherComponent` when this component is first rendered.

`React.lazy` takes a function that must call a `dynamic import()`. This must return a `Promise` which resolves to a module with a default export containing a `React` component.



What is Code Splitting?

Error boundaries

If the other module fails to load (for example, due to network failure), it will trigger an error. You can handle these errors to show a nice user experience and manage recovery with Error Boundaries. Once you've created your Error Boundary, you can use it anywhere above your lazy components to display an error state when there's a network error.

```
import MyErrorBoundary from './MyErrorBoundary';

const OtherComponent = React.lazy(() =>
import('./OtherComponent'));
const AnotherComponent = React.lazy(() =>
import('./AnotherComponent'));

const MyComponent = () => (
  <div>
    <MyErrorBoundary>
      <Suspense fallback={<div>Loading...</div>}>
        <section>
          <OtherComponent />
          <AnotherComponent />
        </section>
      </Suspense>
    </MyErrorBoundary>
  </div>
);
```



What is Code Splitting?

Route-based code splitting

Deciding where in your app to introduce code splitting can be a bit tricky. You want to make sure you choose places that will split bundles evenly, but won't disrupt the user experience.

A good place to start is with routes. Most people on the web are used to page transitions taking some amount of time to load. You also tend to be re-rendering the entire page at once so your users are unlikely to be interacting with other elements on the page at the same time.



What is Code Splitting?

Named Exports

React.lazy currently only supports default exports. If the module you want to import uses named exports, you can create an intermediate module that reexports it as the default. This ensures that tree shaking keeps working and that you don't pull in unused components.

```
// ManyComponents.js
export const MyComponent = /* ... */;
export const MyUnusedComponent = /* ... */;

// MyComponent.js
export { MyComponent as default } from "./ManyComponents.js";

// MyApp.js
import React, { lazy } from 'react';
const MyComponent = lazy(() => import("./MyComponent.js"));
```

Using Typescript with React



TypeScript with React

TypeScript is an object-oriented programming language developed and maintained by the Microsoft Corporation. It is a superset of JavaScript and contains all of its elements.

TypeScript totally follows the OOPS concept and with the help of TSC (TypeScript Compiler), we can convert Typescript code (.ts file) to JavaScript (.js file)





Typescript with React

History of TypeScript

In 2010, Anders Hejlsberg (the creator of TypeScript) started working on TypeScript at Microsoft and in 2012 the first version of TypeScript was released to the public (TypeScript 0.8). Although the release of TypeScript was praised by many people around the world, due to the lack of support by major IDEs, it was not majorly adopted by the JavaScript community.

The First Version of TypeScript (TypeScript 0.8) Released to the Public October 2012.



Typescript with React

Why Should We Use TypeScript?

- TypeScript simplifies JavaScript code, making it easier to read and debug.
- TypeScript is open source.
- TypeScript provides highly productive development tools for JavaScript IDEs
- TypeScript makes code easier to read and understand.
- With TypeScript, we can make a huge improvement over plain JavaScript.
- TypeScript gives us all the benefits of ES6 (ECMAScript 6), plus more productivity.
- TypeScript can help us to avoid painful bugs that developers commonly run into when writing JavaScript by type checking the code.
- Powerful type system, including generics.
- TypeScript is nothing but JavaScript with some additional features.
- Structural, rather than nominal.



Typescript with React

Why Should We Use TypeScript? (continued)

- TypeScript code can be compiled as per ES5 and ES6 standards to support the latest browser.
- Aligned with ECMAScript for compatibility.
- Starts and ends with JavaScript.
- Supports static typing.
- TypeScript will save developers time.
- TypeScript is a superset of ES3, ES5, and ES6.



Typescript with React

Features of TypeScript

- Functions with optional parameters.
- Functions with REST parameters.
- Generics support.
- Modules support.



TypeScript with React

TypeScript includes a very good set of Object Oriented Programming (OOP) features, that are good to maintain robust and clean code; this improves the code quality and maintainability. These OOP features make TypeScript code very clean and organized.

```
class CustomerModel
{
    customerId: number;
    companyName: string;
    contactName: string;
    country: string;
}
class CustomerOperation{
    addCustomer(customerData: CustomerModel) : number {
        //add customer
        let customerId =5; // Id returned after save
        return customerId;
    }
}
```



Typescript with React

To Install typescript with React App

```
npx create-react-app my-app --template typescript
```

To add TypeScript to an existing Create React App project, first install it:

```
npm install --save typescript @types/node @types/react @types/react-dom @types/jest
```

Next, rename any file to be a TypeScript file (e.g. src/index.js to src/index.tsx) and restart your development server!

Resources

<https://reactjs.org/docs/forms.html>

<https://www.sitepoint.com/react-router-complete-guide/>

<https://www.techiediaries.com/react/react-router-5-4-tutorial-examples/>

<https://flaviocopes.com/single-page-application/> <https://reactjs.org/docs/code-splitting.html> <https://dzone.com/articles/what-is-typescript-and-why-use-it>

The background is a solid light blue color. In the bottom-left corner, there is a decorative geometric pattern consisting of several overlapping triangles and squares in various shades of blue, ranging from a very dark navy blue to a medium blue.

Questions?